

Vigil - Technology Stack, Differentiation & Future Scope

Smart India Hackathon 2026 - Problem Statement 26189

AI-Powered Criminal Network Analysis System

GitHub: github.com/SiddharthDC786/investigation-ai

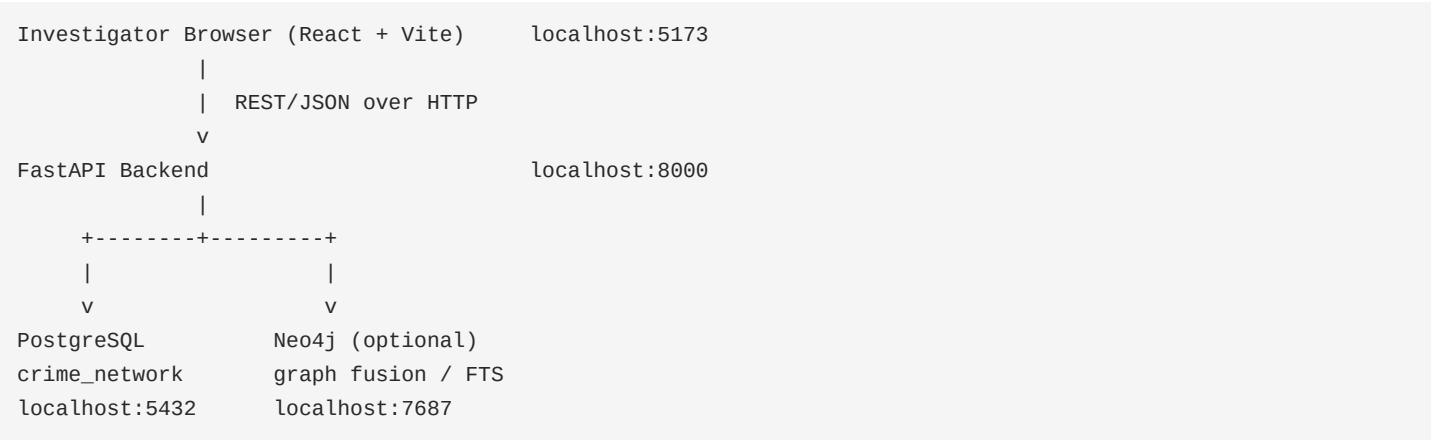
Purpose: Shareable document for judges, mentors, and stakeholders - explains every tool we used, why we chose it, how it powers Vigil, how we differ from commercial systems, user benefits, and our roadmap.

1. Executive Summary

Vigil is a court-defensible criminal investigation console for police and cyber-crime units. It fuses FIR documents, call detail records (CDR), bank transactions, surveillance notes, and lawful OSINT into one searchable system - with explainable AI, not black-box chatbots.

We built Vigil on an open-source stack (React, FastAPI, PostgreSQL, spaCy, NetworkX) so a district cyber cell can run the full product on a laptop - without crore-scale licenses for Palantir Gotham, IBM i2, or Social Links.

2. System Architecture



When the frontend sets VITE_API_BASE_URL=http://localhost:8000, search, timeline, risk scores, ingest, and OSINT hit real PostgreSQL - not mock data.

3. Frontend Technology Stack

Technology	Used for what	Why we chose it	Importance in Vigil
React 19	All UI views: Person Search, Connection Map, Timeline, Priority List, OSINT, Action Log	Industry standard; fast re-renders for live search and dashboards	Core of investigator experience - every screen investigators touch
TypeScript	Typed API responses, props, and state	Catches integration bugs before demo day	Prevents frontend/backend contract mismatches on search, timeline, risk APIs

Vite 8	Dev server and production bundle	Fast hot reload during hackathon iteration	Team could ship features quickly without slow rebuild cycles
Tailwind CSS v4	Dark investigation-console theme	Consistent police-grade UI without custom CSS sprawl	Professional look for judges; readable on long investigation
react-force-graph-2d	Connection Map - persons, phones, weighted call edges	Interactive network layout for suspect-centered stories	Makes hidden CDR links visible - key demo moment for judges
Recharts	Priority List horizontal bar chart	Clear evidence-based ranking for non-technical officers	Supervisors see who to investigate first at a glance
Framer Motion	Subtle panel and tab transitions	Polished UX without heavy animation libraries	Signals production-quality product, not a prototype
IBM Plex fonts	Sans + monospace typography	Readable on data-heavy screens (IDs, phone numbers, hashes)	Reduces eye strain when reading long audit logs and dossiers
i18n (English + Hindi)	Bilingual labels and hints	SIH requirement; usable across Indian states	Officers in Hindi-speaking districts can use the same tool

Where Each Frontend Component Lives

File	Role in project
`SearchView.tsx`	Live person search, filters, disambiguation, matched-person card
`NetworkGraphView.tsx`	CDR-based connection map with evidence on links
`TimelineView.tsx`	Chronological events filtered by **selected person only**
`RiskScoringView.tsx`	Evidence-based priority chart and ranked suspect list
`OsintView.tsx`	Person dossier + lawful record lookups
`AuditTrailView.tsx`	Action log with category badges and hash-chain status
`IngestPanel.tsx`	FIR OCR -> editable text -> preview -> upload to database
`InspectorPanel.tsx`	Selected entity detail + AI explainability panel
`api/*.ts`	HTTP clients calling FastAPI endpoints

Why web, not desktop? Investigators already use browsers. No extra install on every PC - only backend + database on a district server or laptop.

4. Backend Technology Stack

Technology	Used for what	Why we chose it	Importance in Vigil
Python 3.12	Server logic, NLP, graph analytics, ingest	Best ecosystem for data science + police analytics in one language	One team can own API, AI, and analytics without splitting stacks
FastAPI	REST API: `/search`, `/timeline`, `/ingest`, `/analyze`, `/osint`	Auto OpenAPI docs, async-ready, fast router development	Every frontend feature maps to a documented endpoint judges can curl
Uvicorn	ASGI server hosting FastAPI	Production-grade standard for FastAPI	Reliable local and Docker deployment
SQLAlchemy 2	DB sessions and ORM patterns	Reliable PostgreSQL access alongside raw SQL for complex joins	Powers search, timeline, and ingest without ORM-only limitations

psycpg 3	PostgreSQL driver	Modern async-capable driver	Connects to `crime_network` DB with performance headroom
Pydantic	Request/response validation	Strict API schemas	Prevents bad ingest data from corrupting investigation records

Backend Service Map

Service	Used for what	Importance in Vigil
`investigation_search.py`	Fuzzy name search, filter intersection, related people	Solves same-name confusion - core SIH differentiator
`case_graph.py`	Auto connection map from CDR +	Replaces hours of manual i2 link-chart
`timeline_service.py`	Merges CDR, transactions, FIR, chat by	One chronological story instead of five Excel
`risk_score_service.py`	PageRank + role + CDR/FIR evidence scoring	Triage: who to investigate first
`ingest_service.py`	FIR ingest, entity resolution, DB merge	New evidence enters the system without re-importing everything
`ocr_service.py`	FIR photo -> cleaned text (Tesseract)	Officers upload scanned FIRs from the field
`osint_enrichment.py`	Lawful lookup simulation + audit chain	OSINT workflow with legal accountability
`explain_service.py`	Citations and reasoning for selected entity	Court-defensible AI - every flag has a source
`audit_chain.py`	SHA-256 tamper-evident log	Proves OSINT actions were not altered after the fact

Why FastAPI over Django? Lighter weight; each feature is a focused router - ideal for hackathon speed and clear separation of concerns.

5. Database & Storage Stack

Technology	Used for what	Why we chose it	Importance in Vigil
PostgreSQL 16	People, phones, CDR, transactions, FIR, relationships,	ACID compliance; SQL joins; court-grade persistence	Primary evidence store - police IT already runs Postgres
Neo4j 5.26 (optional)	Graph fusion, cross-source full-text fallback	Graph-native linking for advanced demos	Shows graph expertise; app works fully without it

Key Tables and Why They Matter

Table	Used for what	Importance in Vigil
`people`	Canonical identity (name, DOB, gender, city)	Ground truth for every search and timeline
`recorded_names`	FIR typos, aliases, maiden names	Fuzzy search works on "Mukherjee" not just "Mukherjee"
`phones`	Person ? phone links	Joins identities to CDR for connection map
`cdr`	Call records	Powers graph edges and timeline call events
`bank_accounts` + `transactions`	Money flow	Financial crime timeline and handler detection
`fir`	Complaint text	NLP ingest and FIR mention scoring in priority list

`relationships`	Handler, associate, witness roles	Role-aware disambiguation and risk scoring
`audit_chain`	Tamper-evident OSINT log	Court disclosure and supervisor oversight

6. AI, NLP & Analytics Stack

Technology	Used for what	Why we chose it	Importance in Vigil
<code>**spaCy (en_core_web_sm)**</code>	Extract PERSON, ORG, GPE from FIR on ingest	Deterministic NER - citeable in court	Auto-links FIR names to people records
<code>**Regex patterns**</code>	Indian phones (+91), bank accounts, vehicles	High precision on structured IDs	Catches identifiers spaCy might miss
<code>**RapidFuzz**</code>	Fuzzy name match in search + ingest	Solves typos without opaque ML	Core identity-resolution engine
<code>**NetworkX**</code>	PageRank, betweenness, Louvain communities	Standard graph algorithms	Finds hubs and criminal rings automatically
<code>**SciPy**</code>	Sparse matrix support for centrality	Efficient on 60K+ CDR rows	Priority list scales to real case volume
<code>**Pillow + Tesseract**</code>	OCR: FIR photo -> editable text	Field officers upload scans	Bridges paper FIR world to digital investigation

Why Not ChatGPT for Everything?

Court testimony requires citations - e.g. "matched recorded_names at 92% confidence." Black-box LLM answers are hard to defend. Vigil uses transparent pipelines with /explain endpoints and structured audit logs. Works offline - no cloud API required.

7. Data Pipeline & DevOps

Component	Used for what	Why we chose it	Importance in Vigil
<code>**generate_dataset.py**</code>	240 cases, 60K CDR, 25K transactions (synthetic)	Safe demo without real PII	Judges see realistic scale without privacy risk
<code>**import_to_db.py**</code>	Load CSVs into PostgreSQL	One-command seed	Teammates reproduce demo in minutes
<code>**seed_demo_rahuls.sql**</code>	Multiple "Rahul" people in CASE0001	Name disambiguation demo	Proves same-name resolution live
<code>**Docker Compose**</code>	Postgres + Neo4j + backend containers	One-command setup	Onboarding new team members and judges' machines
<code>**GitHub**</code>	Version control and collaboration	Standard team workflow	github.com/SiddharthDC786/investigation-ai

8. Feature-to-Technology Map

App feature	Technologies involved	Why this combination
Person Search	React, FastAPI, PostgreSQL, RapidFuzz, `recorded_names`	Fuzzy + alias table solves Indian FIR spelling chaos
Connection Map	react-force-graph-2d, `case_graph.py`, CDR	Visual proof of hidden phone bridges
Event Timeline	`TimelineView`, `timeline_service.py`	Person-filtered fusion - not a dump of all case noise
Priority List	Recharts, NetworkX, `risk_score_service.py`	Evidence-weighted triage, not gut feeling
Record Search (OSINT)	`OsintView`, `explain_service.py`, `audit_chain.py`	Enrichment + accountability in one flow
FIR Upload	Tesseract, spaCy, `ingest_service.py`	Human-in-the-loop: OCR -> edit -> then DB
Action Log	`AuditTrailView`, SHA-256 chain	Tamper-evident trail for court
Explainability	`explain_service.py`, DB citations	Every AI flag is defensible

9. How Vigil Is Different From Existing Tools

Commercial investigation platforms inspire our design - but Vigil solves the same problems without crore licenses, vendor lock-in, or black-box AI.

vs Palantir Gotham (Data Fusion)

Gotham limitation	How Vigil is different
Crore-scale licensing	Open source: PostgreSQL, spaCy, Neo4j Community - zero license cost
Closed-source linking	RapidFuzz threshold (88%) is documented and inspectable
Requires dedicated analysts + weeks of training	Web UI; investigator searches in minutes
Cloud / server farm dependency	Runs on a district laptop , offline-capable
No built-in FIR OCR for field uploads	Upload scanned FIR -> OCR -> edit -> ingest pipeline

vs IBM i2 Analyst's Notebook (Link Analysis)

i2 limitation	How Vigil is different
Manual link charts drawn by hand	Auto-generated graph from CDR + relationships
Desktop-only legacy UI	Web-based React - any district PC with a browser
Per-seat licensing	Open stack, self-hosted
No same-name resolution	Disambiguation screen before showing network
Static snapshots	Re-ingest FIR/CDR -> graph and timeline update
No hidden-bridge detection	Finds shared prepaid SIMs with no direct CDR between suspects

vs Social Links / OSINT SaaS

Social Links limitation	How Vigil is different
Cloud SaaS - data leaves police network	**Self-hosted** - all data on police infrastructure
Opaque enrichment sources	Every lookup logged with operator ID + lawful basis
Audit trail can be disputed	**SHA-256 hash chain** - tampering breaks <code>`/audit/verify`</code>
No graph integration	OSINT results link back to suspect nodes in connection map

vs Excel + Manual Police Workflows

Manual workflow	How Vigil is different
Five separate Excel files	**One person-centric screen**
Ctrl+F on exact spelling	**Fuzzy search** + alias table
No graph view	Interactive connection map
No audit of lookups	Hash-chained action log
"AI" = asking ChatGPT about FIR	**Cited <code>`/explain`</code> from database records

Our Unique Differentiator: Court-Defensible Explainability

Feature	What it gives investigators
<code>`/explain/{person_id}`</code>	Plain-language reasoning + source citations
<code>`/analyze/risk-score`</code>	Why this person scored 87/100 (centrality + role + CDR/FIR)
<code>`/case-summary/{case_id}`</code>	Briefing narrative for senior officer
Template-based narratives	Every sentence traceable to a DB row - **not ChatGPT**

10. How Vigil Is Useful for Users

Who Uses Vigil

User	Role	How Vigil helps them
Field investigator (IO)	First-line case work	Search, disambiguate names, pick correct suspect in minutes
Cyber crime analyst	CDR / digital forensics	Call graph, shared-contact bridges, person-filtered timeline
OSINT officer	Lawful public lookups	Person dossier + every lookup logged for disclosure
Case supervisor	Oversight and court prep	Verify audit hash chain; export JSON for disclosure
Senior officer / judge	Briefing and evaluation	Priority list, case summary, end-to-end demo story

Concrete User Benefits

1. ****Same-name resolution**** - Search "Rahul" + city + phone; choose suspect vs witness vs handler instead of guessing.

- 2. **Multi-source fusion** - One timeline (calls, money, FIR) filtered by the person you selected.
- 3. **Hidden links exposed** - Shared prepaid SIM bridge when suspects never call each other directly.
- 4. **Evidence-based priority** - Rank suspects by graph centrality + role + CDR/FIR mentions, not intuition.
- 5. **FIR ingest with human check** - OCR fills editable text; officer verifies before database save.
- 6. **Court-defensible audit** - Every OSINT action in a tamper-evident chain with officer badge ID.
- 7. **Explainability** - Every flag cites `people`, `recorded\_names`, CDR count, or FIR text.
- 8. **Offline capable** - No OpenAI or cloud API; runs in district cyber cells without internet.

Primary Persona

A district cyber cell officer with FIR PDFs, Excel CDR exports, and bank CSVs who today switches between five tools - and with Vigil gets one investigation console on a single screen.

Demo Story (CASE0001)

Search Rahul Mukherjee -> disambiguate Hyderabad suspect (P00014) -> Connection Map shows CDR network -> Timeline shows only his events -> Priority List ranks evidence -> OSINT dossier -> Action Log proves every step -> Supervisor exports audit JSON.

11. Current Limitations (Honest Assessment)

Area	Current status	Why it matters
Demo auth	Client-side login only	Production needs JWT / government SSO
Face search	Fixed demo match	Real deployment needs biometrics API + legal framework
OSINT sources	Synthetic enrichments	Production connects to licensed government APIs
Neo4j	Optional, often off in demo	Full graph fusion needs Docker bundle
Languages	English + Hindi	Regional languages needed for all states
Mobile	Responsive web only	Field officers may need native mobile app

Judges respect teams who know their limits - and have a plan to fix them.

12. Future Scope & Roadmap

Phase 1 - Production Hardening (0-6 months)

Improvement	Technology / approach	User impact
JWT + RBAC auth	OAuth2, role-based FastAPI dependencies	Real officer accounts; supervisor vs IO permissions
Government SSO	Integration with state police identity systems	Single sign-on across departments
Real data connectors	ETL pipelines for CDR CSV, bank feeds, FIR PDF bulk import	Replace synthetic CSV with live case data
Neo4j always-on	Docker Compose production profile	Full graph fusion without manual setup
Automated backups	PostgreSQL pg_dump + audit chain export	Disaster recovery for active investigations

Phase 2 - Enhanced Intelligence (6-12 months)

Improvement	Technology / approach	User impact
Multilingual NER	Hindi/regional spaCy or IndicBERT models	Extract entities from FIR in local languages
Real OSINT APIs	MCA, court bulletins, vehicle RTO (with legal authorization)	Live public-record enrichment
Real face matching	Integrate NCRB / state biometrics API	Photo-to-suspect identification with audit
Alert rules engine	New CDR ingest triggers "suspect called handler" alerts	Proactive investigation instead of reactive search
Community detection UI	Louvain clusters rendered as separate ring views	Visualize criminal cells automatically

Phase 3 - Scale & Collaboration (12-24 months)

Improvement	Technology / approach	User impact
Multi-case dashboard	Cross-case person linking (same phone in two FIRs)	Detect serial offenders across districts
Inter-agency federation	Encrypted case sharing between state cyber cells	National network crime coordination
Mobile field app	React Native or PWA with offline queue	IOs capture FIR photos and notes in the field
Advanced timeline	Geo-map of cell-tower locations from CDR	Spatial movement analysis
ML-assisted triage (explainable)	Gradient boosting on features with SHAP explanations	Smarter priority list - still citeable

Phase 4 - Policy & Ecosystem (24+ months)

Improvement	Technology / approach	User impact
State-wide deployment	Kubernetes + air-gapped on-prem cluster	Entire state police on one Vigil instance
Training simulator mode	Synthetic case generator for academy	Train new IOs without real PII
Open API for courts	Read-only disclosure endpoints with audit	Faster court submissions
Integration with NCRP / CCTNS	Standard police RMS connectors	Vigil as analytics layer on existing records
Regional language UI	Tamil, Telugu, Marathi, Bengali i18n	Pan-India adoption

Long-Term Vision

Vigil becomes the default open-source investigation console for Indian cyber cells - delivering Gotham + i2 + Social Links workflows at district-PC cost, with explainable AI and tamper-evident audit built in from day one. Every conclusion

an investigator presents in court can be traced to a database record, a fuzzy-match score, or a hash-chained OSINT log entry.

13. Quick Reference

Item	Value
Frontend	http://localhost:5173
Backend API	http://localhost:8000
PostgreSQL	localhost:5432 / database: `crime_network`
Neo4j (optional)	http://localhost:7474
Login	INV-2847 / vigil2026
Demo case	CASE0001 (FIR-042/2026)
Main suspect	P00014 - Rahul Mukherjee, Hyderabad
GitHub	github.com/SiddharthDC786/investigation-ai

14. Elevator Pitch

Vigil is an open-source criminal investigation console for Smart India Hackathon PS 26189. Built with React, FastAPI, PostgreSQL, spaCy, RapidFuzz, and NetworkX, it fuses FIR documents, call records, and bank data into one searchable system - solving same-name confusion, FIR typos, and hidden phone bridges. Unlike crore-priced commercial tools, Vigil runs on a district laptop, explains every flag with citations, and logs every OSINT action in a tamper-evident audit chain. Our roadmap extends this to production auth, real government data feeds, multilingual FIR processing, and statewide deployment - making advanced network investigation accessible to every cyber cell in India.

Vigil Team - Smart India Hackathon 2026 - Shareable documentation